ResilienceAI

AI-driven chaos engineering for cloud-native resilience

Project deep dive: everything that was built, why each decision was made, every problem we hit and how it was fixed, how every number is calculated, and what to say to the judges.

Hackathon track: AI-Driven Chaos Engineering for Cloud-Native Resilience (BGSW / Bosch).

Section	What it covers
1	The big picture in plain language
2	Day 1 requirements and Day 2 requirements, and how each is met
3	The build journey, step by step (environment, cluster, shop, bugs)
4	Architecture: every component and how they talk to each other
5	Fault injection: the three error types, exactly how they are done
6	Telemetry: how we measure what shoppers experience
7	Safety: validation, health gate, kill switch, rollback, guardrail, backstop
8	The resilient shop frontend (the Go patch) and the isolation results
9	The AI: single planner, then the 3-agent squad, models, randomisation
10	Severity ranking, resilience score and fix recommendations, with worked maths
11	The user interfaces: Streamlit, built-in dashboard, React control center
12	Technology stack and frameworks
13	Problems we hit and how each was solved (full log)
14	Results, limitations and next steps
15	Demo script and judge Q&A preparation
16	Glossary

1. The big picture

Modern apps are built from many small programs (**microservices**) that call each other over the network. An online shop might have one service for the product list, one for the cart, one for payments, one for currency conversion and so on. When one of them fails, the failure can spread in unexpected ways: a broken advert service should never stop someone from buying, but in many real systems it does.

Chaos engineering means breaking things on purpose, in a controlled way, to find these weak points before a real outage does. Netflix made it famous with 'Chaos Monkey'. The usual cycle is: measure normal behaviour (baseline), state a hypothesis, inject a fault, measure the impact, roll back, learn.

Our idea: let AI agents choose which faults to inject, keep them inside strict safety limits, measure the impact on real (simulated) shoppers, and have a separate AI rank the damage and recommend fixes. Everything runs on one laptop, on a real Kubernetes cluster, with a real 11-service shop.

One-sentence pitch: Three AI agents on three different models each break one part of a real microservices shop inside a sandbox; a safety controller keeps it controlled, live telemetry measures what shoppers feel, an analyst model ranks the damage and recommends fixes, and one click rolls everything back with verified recovery.

What is running on the laptop

Piece	What it is	Address
The shop	Google Online Boutique, 11 microservices + Redis, on Kubernetes	http://localhost:8080
Backend	Our Python FastAPI server: safety controller, agents, telemetry, analysis	http://127.0.0.1:8000
Control center	Our React web app (what you present)	http://localhost:5173
AI models	Ollama running 4 local models on the laptop GPU	http://localhost:11434
Cluster	kind (Kubernetes inside Docker), cluster name chaos-sandbox	kubectl context kind-chaos-sandbox

2. Hackathon requirements and how we meet them

Day 1

Requirement	How we meet it
Running app	Online Boutique deployed in namespace 'boutique' on a kind cluster; shop on port 8080
Dependency map	Built automatically from the Kubernetes manifests (every *_ADDR environment variable is an edge) plus a small overrides file; shown live in the control center
Baseline	60 s of healthy synthetic traffic: 1,940 requests, 0% errors, p50 12.8 ms, p95 26.1 ms (saved in out/baseline.json)
One safe failure injection	Outage, crash loop or network delay, validated against an allowlist, injected only into a healthy shop
Kill switch	Freezes the system instantly in its current state (button, API, CLI, or the file out/STOP)

Day 2

Requirement	How we meet it
AI chooses / ranks experiments	3 agents on 3 different models choose their faults; an analyst model ranks the resulting errors by severity
3+ fault types	service-down (outage), pod-kill (crash loop), network-delay (latency)
AI findings report	Findings page: resilience score, ranked findings, measured impact, suspected cause, AI fix recommendations; export to Markdown

Requirement	How we meet it
Deliverables	Source + README, demo video (3 min), architecture overview (README + live map), findings report, team form

3. The build journey, step by step

3.1 Setting up the laptop (Windows 11)

- The first attempt used **brew**, which is a macOS tool and does not exist on Windows. We switched to **winget**, the Windows package manager, to install Docker Desktop, kind, kubectl, Python, Node.js and Ollama.
- Docker Desktop needs **WSL 2** (Windows Subsystem for Linux). *wsl --install* hung silently with no output on a slow network. Fix: download the WSL MSI installer directly, verify it, and install it with admin rights, then reboot. After that Docker Desktop started normally.
- A Python virtual environment (*.venv*) holds the backend's packages (requirements.txt).

3.2 Creating the cluster and deploying the shop

- **kind** creates a whole Kubernetes cluster inside one Docker container (*chaos-sandbox-control-plane*). Script: *setup-cluster.ps1*.
- The shop is deployed from Google's release manifest into namespace **boutique**. First image pulls take 5-10 minutes.
- *open-shop.ps1* runs *kubectl port-forward* in a loop so the shop is always reachable on port 8080.
- Baseline captured with *chaos.cmd baseline* (60 s of synthetic shoppers).

3.3 First bug we found: no December

On the checkout form, the card-expiry **month dropdown had January twice and no December**. That is a bug in Google's template. We fixed the month list in *templates/cart.html* (12 months, values 1-12). At first we applied it without rebuilding the image, by mounting a corrected template from a Kubernetes ConfigMap (*patches/apply-frontend-patches.ps1*); later the fix was baked into our own frontend image.

3.4 First manual error: the cart empties on refresh

We scaled **redis-cart** (the cart's database) down and back up. Every shopper's cart was emptied. Reason: redis-cart stores data in an *emptyDir* volume, which lives only as long as the pod. A restart means data loss. That was our first real resilience finding: **the cart has no persistence**.

3.5 Building the chaos engine (backend)

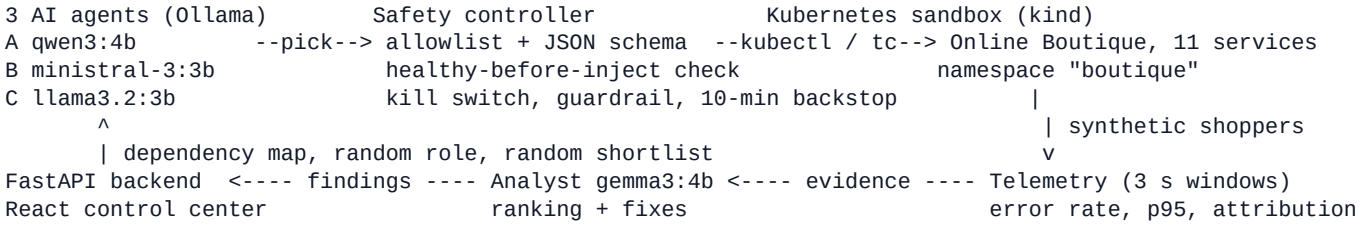
- **Discovery** (discovery.py): reads the manifests and builds the dependency map.
- **Fault templates** (faults.py): the only code that changes the cluster.
- **Safety controller** (controller.py): validate, health check, inject, monitor, stop, clean up, verify recovery.
- **Probe** (probe.py): synthetic shoppers. **Telemetry** (telemetry.py): 3-second windows and error attribution.
- **Planner** (planner.py): the first AI agent, using a local model through Ollama.
- **API** (api.py): FastAPI endpoints for the dashboards.

3.6 The order things were added, and why

Step	What changed	Why
1	Single AI planner (qwen3:4b) ranking experiments	Day 1 needed an agent that injects errors
2	Streamlit dashboard connected to the backend	Your teammate's app.py had a demo mode only
3	Hold mode: no duration slider, faults stay until Rollback	You asked for errors to persist until rollback
4	Kill switch = freeze; Rollback = restore initial state	Your definition: stop at last state vs back to start

Step	What changed	Why
5	AJAX dashboard served by the backend	Streamlit flickered on every refresh; counts were confusing
6	Ranking removed; random single-fault injection	You asked for random errors, 1 at a time
7	Resilient Go frontend + isolation test	One broken service crashed the whole shop
8	3-agent squad, analyst ranking and fixes	Day 2: multiple agents, ranking, recommendations
9	React control center	Professional UI to replace Streamlit
10	Randomised roles/shortlists; findings only for the active run	Agents repeated the same picks; old findings lingered
11	Live dependency map on every page; Gemma 3 analyst	Map was hidden; cloud model returned 401
12	Label consistency, minimum sample size, emergency guardrail	Labels contradicted scores; safety net needed

4. Architecture



4.1 Backend modules (folder chaos/)

File	Responsibility
api.py	All HTTP endpoints, the one-job-at-a-time scheduler, rollback, the background fault watcher (refreshes active faults every 3 s so /status answers in about 9 ms), the emergency guardrail thread, CORS for the React app
squad.py	The 3-agent run: random roles, random shortlists, model calls, validation, health gate, injection, hold loop, kill-switch freeze, backstop, triggers analysis
severity.py	Measures each error's impact from telemetry, computes the formula score, asks the analyst to rank, enforces consistency
fixes.py	Asks the analyst for a fix per error (cause, immediate fix, long-term fix, prevention, confidence)
analyst.py	Chooses the analyst model (gemma3:4b, fallback qwen3:4b) and explains why if it falls back
llm.py	Streams a chat to Ollama with a JSON schema that constrains the answer; random seed support
controller.py	Validation gate; the single-experiment runner with steady-state check, monitoring, abort thresholds, cleanup and recovery check
faults.py	Fault templates (ServiceDown, PodKill, NetworkDelay), find_faults() and reset_all() for rollback
telemetry.py	Ingestor: continuous probe traffic, 3 s windows, per-journey stats, error attribution to services
probe.py	The synthetic shopper and its timing statistics
discovery.py / overrides.yaml	Dependency map from manifests plus manual corrections (criticality, journeys)
safety.yaml	Every limit and threshold in one file
planner.py / run.py	The original single planner and the command-line tool (chaos.cmd)

4.2 Ports and connections

Port	Used by
8080	The shop (kubectl port-forward to the frontend service)
8000	Backend API and the built-in dashboard; /docs is the interactive API documentation
5173	React control center (Vite dev server)
8001	kubectl proxy used by single experiments' own probe
8002	kubectl proxy used by continuous telemetry ingestion
11434	Ollama model server

Why kubectl proxy for the probe? It goes through the Kubernetes API server to the frontend *Service*, so it keeps working even while pods are being killed and replaced, unlike a port-forward to one pod.

5. Fault injection: exactly how each error is made

Fault	What we do	What it simulates	Undo
service-down (Outage)	kubectl scale deployment X --replicas=0	A service completely gone (bad deploy, node loss)	Scale back to the replica count in the manifest
pod-kill (Crash loop)	Force-delete the pod; while held, a background loop kills every replacement pod every 1 s	A service that keeps crashing	Stop the loop; Kubernetes recreates the pod
network-delay (Latency)	tc qdisc ... netem delay N ms on the pod's host-side network interface (veth), run with docker exec inside the kind node	A slow dependency (overloaded service, bad network)	Delete the netem rule

Why the crash loop needed fixing

At first a crash was a single pod kill. Go services restart in about 1 second, so the 'crash' flickered: sometimes the shop worked, sometimes it did not. The fix was a background thread that kills replacement pods every second while the fault is held, so the outage is continuous until rollback. On kill switch, the loop stops and the deployment is pinned at 0 replicas, so the frozen state really stays crashed.

Why network delay is applied on the veth

Every pod has a virtual network cable (veth pair). The host-side end carries traffic *into* the pod, so a netem delay there slows every call into that service, without touching the container image.

How rollback finds everything to undo

find_faults() compares the cluster with the initial state in one pass: every deployment whose replica count differs from the manifest, and every netem rule on a pod interface (one kubectl call for deployments, one for pods, and two commands per node). *reset_all()* undoes each one. This means rollback works even after a backend restart, because it looks at the real cluster, not at memory.

6. Telemetry: measuring what shoppers feel

6.1 The synthetic shopper

- 2 workers, 0.5 s pause between journeys, 5 s timeout per request.
- Journey mix 6 : 3 : 1, roughly like real traffic: **browse** (home, then a product page), **cart** (add to cart, then view cart), **checkout** (add to cart, then place order with a test card).
- Each request records: time, journey, step, HTTP status, latency, success, the error text from the shop's error page, and the **X-Degraded** header (which services a page had to do without).

6.2 Three-second windows

Every 3 s the ingestor computes for the previous window: request count, error rate, p50 and p95 latency, per-journey error rate and p95, and how many requests are still **hanging**. It reads one window behind so slow requests have time to finish. Hanging requests matter: without counting them, a dependency that hangs looks like 'no data' instead of 'very slow'.

p95 latency = the time under which 95% of requests finished. It shows what the slowest normal users feel, which the average hides.

6.3 Error attribution: which service caused this?

- **From the error message:** the shop's error page names the failing call (for example 'failed to charge card', 'could not retrieve product', or a port like :3550). Phrase and port rules map that to a service.
- **From the X-Degraded header:** our patched frontend lists the services it had to work around.

- **From the page's dependencies:** if the message gives no clue (for example a slow page), the error is credited to an active fault that this page depends on, preferring the latency fault for slow or timed-out requests.
- **The redis-cart rule:** the frontend only talks to cartservice, so a redis-cart outage looks like a cartservice error. When only redis-cart is under test, the error is credited to redis-cart. Agents are also not allowed to split that pair between them, because nobody outside could tell them apart.

6.4 Failed, slow, degraded

Category	Meaning
Failed	HTTP error, connection error or timeout: the shopper saw an error
Slow	Succeeded but slower than max(2 x baseline p95, 250 ms)
Degraded	Succeeded but showed a fallback (for example prices in USD only, cart badge hidden)

'Active errors' and 'affected services' cover the last 15 seconds, so the numbers on screen describe now, not history. That fixed an early complaint about counts like '35 active errors' that never went down.

7. Safety: every guardrail

Safeguard	Exactly how it works
Sandbox scope	Every command uses the fixed context kind-chaos-sandbox and namespace boutique; anything else is refused
Allowlist	Only the 11 listed services can be targeted; loadgenerator and everything outside the namespace cannot. The agents additionally exclude frontend (the entry point)
Schema-constrained AI	The model's answer must match a JSON schema; for agents, the target is an enum of the shortlisted services, so the model physically cannot name anything else
Validation gate	controller.validate(): known fault type, allowlisted target, parameters as integers within limits (delay 50-3000 ms), duration within caps
One job at a time	The backend refuses a second experiment while one runs, and refuses new faults while old ones are still in the cluster (press Rollback first)
Healthy before injecting	Squad: waits up to 45 s for telemetry to show a healthy shop (error rate <= 2%, p95 <= max(1.5 x baseline, 100 ms)); single experiments run a 15 s steady-state check
Kill switch	Writes out/STOP; every loop checks it every second; faults are frozen in place (crash loops pinned at 0 replicas)
Rollback	Stops everything, reset_all(), waits for all deployments to be ready, then waits for 3+ healthy telemetry windows (up to 120 s) before saying 'recovery verified'
Emergency guardrail	Automatic rollback if 90% of all requests or 95% of browsing fail (or requests hang) continuously for 30 s. Starts about 33 s after a total outage, shop verified healthy about 1 minute after it went down
Timed-experiment abort	Command-line experiments stop by themselves at error rate > 10% or p95 > max(2 x baseline, 250 ms) for 2 windows (6 s), or on telemetry loss, or at 30 s
Backstop	Any held fault is removed automatically after 10 minutes
Audit trail	Every event, run, squad run and error is saved (out/runs, out/squads) and shown in History

Kill switch vs rollback (important to say clearly): the kill switch *freezes* - it stops the agents and leaves the system exactly as it is, so you can inspect it. Rollback *restores* the initial state and proves recovery. They are deliberately separate buttons.

Why the guardrail threshold is so high: the squad's job is to cause damage, and normal runs reach up to about 55% failures. A low threshold would roll back every run within seconds and you could never demonstrate a manual rollback. 90% for 30 s only fires when the shop is essentially dead.

8. The resilient shop frontend

When we injected single faults, **everything crashed**, even unrelated pages, and it was random: sometimes it worked, sometimes not. Three root causes in Google's original Go frontend:

- The cart counter and the currency list are drawn on **every page**. If the cart or currency service failed, the whole page became an error page.
- Calls to other services had **no time limit**, so a slow service made pages hang for about 5 s.
- After an outage, the gRPC client's reconnect **backoff** grew large, so recovery took 30-70 s even after the service was back.

What we changed (microservices-demo/src/frontend)

Change	Detail
Graceful degradation (degrade.go)	Each optional call can fail into a fallback: currencies -> USD only, cart -> empty badge, shipping -> 'unavailable', missing recommendation products skipped; a yellow notice bar explains what is missing

Change	Detail
Required vs optional	Product page requires the catalogue; cart page requires the cart; checkout requires its services. Everything else is optional
Per-call timeouts (rpc.go)	1.5 s default, 0.8 s for currency, 0.5 s for optional features; place order 4 s
Fast reconnect (main.go)	gRPC backoff base 200 ms, max 2 s, minimum connect timeout 2 s
X-Degraded header	Lists the services the page worked around, so telemetry can attribute degradation
December fix	Baked into the image's cart.html

Built as image *frontend-resilient:v1*, loaded into kind (no registry needed). Switch any time with *python patches\use_frontend.py original* or *resilient* to show the before/after.

Isolation results: one service down at a time

Service down	Pages that fail	Pages that keep working
redis-cart	view cart, add to cart, checkout	home, product (cart badge hidden)
currencyservice	checkout	all others, prices in USD
shippingservice	checkout	all others, 'shipping unavailable'
productcatalogservice	product, add to cart, checkout	home (notice), view cart
checkoutservice	checkout	all others
paymentservice	checkout	all others
recommendationservice	none	all (section hidden)
adservice	none	all (advert hidden)
emailservice	none	all (order still placed)

Recovery after every rollback was verified, in 9-33 s. Note for honesty: for cartservice and emailservice our test checked pages 4 s after scaling down, while the old pod (5 s shutdown grace period) was still answering, so those two rows were not fully confirmed by that test.

9. The AI

9.1 We did not train a model

All models are **pre-trained, open-weight** models downloaded through Ollama. Our engineering is in how they are used: the context they get (dependency map, journeys, shortlist), the output format they are forced into (JSON schema), validation of every answer, retries and fallbacks, and measuring the real effect. This is the honest answer if a judge asks 'how did you train it'.

9.2 Phase 1: the single planner

- qwen3:4b with a visible **thinking** phase, temperature 0.2, JSON-schema output.
- Context: dependency map, baseline, fault catalogue with parameter limits, allowlist, previous runs.
- Problems fixed: a 16k context did not fit the 6 GB GPU and spilled to the CPU (5x slower) -> 8k context; the model proposed 300-second experiments -> min/max limits in the schema and the prompt.
- Claude (claude-opus-5-5) is supported if an API key is set; not needed and not used now.

9.3 Phase 2: the 3-agent squad

Role	Model	Maker	Notes
Agent A	qwen3:4b	Alibaba	Thinks before answering (about 45-70 s); its reasoning streams into the UI
Agent B	ministral-3:3b	Mistral AI	Chosen instead of Mistral 7B because the disk had only 4.7 GB free
Agent C	llama3.2:3b	Meta	Already installed; fast (about 6 s)
Analyst	gemma3:4b	Google	Ranks and recommends; separate from the agents. Cloud gemma4:31b optional

How one agent decides

- Its error type for this run is assigned at random (the three types are shuffled across the three agents).
- It receives a random **shortlist of 4 services**, avoiding the previous run's targets when enough remain, and excluding services already taken by another agent (and the linked cartservice / redis-cart pair).
- It receives the services' roles, which are critical, who calls them, and the user journeys.
- Ollama forces the answer into JSON: target (one of the shortlist), hypothesis, reason, and delay for latency.
- Temperature 0.9 plus a random seed per call, so answers vary between runs.
- Invalid answer -> retry once; still invalid -> a random valid pick, clearly flagged 'random fallback'.
- The models run one after another and are unloaded after use (keep_alive 0), because the 6 GB GPU fits one at a time.

Why no agent framework

We did not use LangChain, CrewAI or AutoGen. The squad is about 300 lines of our own Python, so every step is visible, testable and auditable, which matters for a safety-critical tool. Ollama's structured output gives the reliability those frameworks would otherwise add.

Variety, proven

Agent	Dry run 1	Dry run 2
A qwen3	outage on redis-cart	latency on productcatalogservice
B ministral	latency on currencyservice	outage on checkoutservice
C llama3.2	crash loop on shippingservice	crash loop on cartservice

10. Severity, resilience score and fixes

10.1 When analysis starts

After injection the squad observes for at least **30 s** and at least **150 shopper requests** (at most 90 s). The request minimum was added after one run measured only 48 requests: a slow fault makes shoppers slower, and 1-2 failures out of 48 is too small to judge.

10.2 Measuring each error

For each injected error, from its injection time to now: failed, slow and degraded requests attributed to its service; the total number of requests; which pages broke; which journeys failed.

10.3 Formula score (0-100) per error

Factor	Weight	Calculation
User impact	40	min(1, failure rate / 0.25) - 25% or more failures = full marks
Business impact	25	worst broken journey: checkout 1.0, cart 0.7, browse 0.6; if nothing failed but requests were degraded or slow: 0.15
Blast radius	20	pages broken / 5 (home, product, view cart, add to cart, checkout)
Slowdown	15	min(1, slow rate / 0.25)

Labels: **70+ Critical, 40-69 High, 20-39 Medium, under 20 Low**. The label always comes from the score, so the label, bar and resilience score never contradict each other. (Before this fix the Gemma analyst called a score of 36 'Critical'.)

10.4 The analyst's ranking

The analyst receives the measurements and formula scores and returns an order with a justification per error. It must rank exactly these errors, each once. It may reorder errors whose scores are within 10 points (it can weigh things the formula cannot), but it may not put an error above one that measured clearly worse; if it tries, its explanations are kept and the measured order is used, with a note. If its answer is invalid, the formula ranking is used and the UI says so.

10.5 Resilience score

```
resilience = 100 - (0.6 x worst error score + 0.4 x average error score)
```

It answers: **how well did the shop keep working for shoppers while this run's errors were active?** 100 = nobody noticed; 0 = as bad as the scale measures. The worst error dominates, because one broken checkout hurts users regardless of how well everything else held. It describes this run's errors, so compare runs with similar targets or average several runs.

Worked example 1 (score 46)

Error	Measured	Points	Score
productcatalogservice outage	102/187 failed (54.5%), checkout journey broken, 2 pages broken	40 + 25 + 8 + 0	73
cartservice crash loop	0 failed, 61 degraded	0 + 3.75 + 0 + 0	4
checkoutservice +2500 ms	nothing measured	0	0

```
average = (73 + 4 + 0) / 3 = 25.67
resilience = 100 - (0.6 x 73 + 0.4 x 25.67) = 100 - (43.8 + 10.27) = 45.93 -> 46
```

Worked example 2 (score 71)

```
scores 36, 22, 0 -> average 19.33
resilience = 100 - (0.6 x 36 + 0.4 x 19.33) = 100 - (21.6 + 7.73) = 70.67 -> 71
```

Masking insight: in example 1, the checkout slowdown scored 0 because the catalogue outage stopped shoppers at 'add to cart', so they never reached checkout. One failure hid another - exactly the kind of interaction chaos engineering exists to reveal.

10.6 Fix recommendations

For each error, in rank order, the analyst gets the fault, the service, the measured impact and sample error messages, and must return: **likely cause, immediate fix, long-term fix, prevention, confidence**. They are advice for people and never executed. Example (paymentservice outage, Gemma 3, 13 s): immediate fix 'scale the deployment back'; prevention 'readiness probes and automatic scaling'.

11. The user interfaces

UI	Status	Notes
Streamlit (dashboard/app.py)	Fallback, unchanged	Your teammate's original app; flickered because Streamlit reruns the whole page on every refresh
Built-in dashboard (:8000)	Fallback	Plain HTML + JavaScript polling the API every 2 s (AJAX), no flicker
React control center (:5173)	Main UI	Professional console, all features

React control center

Area	Contents
Every page	Live service dependency map (healthy / chosen by agent / fault injected / affected, live failed-slow-degraded counts, status line), backend connection, Kill switch, Roll back
Overview	KPIs vs baseline, current experiment and guardrail state, telemetry start/stop, random injection, the 3-agent squad, severity ranking and fix recommendations, manual experiment, charts with fault windows shaded, affected services
Experiments	Same controls, focused on running experiments
Findings	Resilience score, ranked findings, measured impact, suspected cause, recommended fixes, export report
History	Events, experiments, squad runs, errors: sortable and filterable

- Polls the backend every 2 s; every panel has loading, empty and error states; a toast for every action.
- Shows only data the backend returned: no placeholder numbers. 'Recovery verified' only when the backend says so.
- Findings and agent picks appear only for the run in progress; after rollback they clear and move to History.

12. Technology stack and frameworks

Layer	Technology	Where / how
Target app	Google Online Boutique (Go, C#, Node.js, Python, Java, gRPC)	The system under test; we patched the Go frontend
Infrastructure	Docker Desktop + WSL 2, kind, Kubernetes, kubectl, Linux tc netem	Sandbox cluster and fault injection
Backend	Python, FastAPI, Uvicorn, Pydantic, Requests, PyYAML	api.py endpoints; Pydantic models define request bodies and the AI output formats
AI runtime	Ollama with JSON-schema structured output	llm.py; 4 local models
Frontend	React 18, TypeScript, Vite, Tailwind CSS 4, Recharts, React Flow, lucide-react	frontend/; charts, dependency map, icons
Fallback UIs	Streamlit + pandas; plain HTML/JS	dashboard/app.py; chaos/static/dashboard.html

13. Problems we hit and how each was solved

Problem	Cause	Fix
brew not found	brew is macOS-only	Used winget
wsl --install hung with no output	Slow network download inside the installer	Downloaded and installed the WSL MSI directly, rebooted
'chaos.run' not recognised	Ran from the wrong shell / without the venv	chaos.cmd wrapper that uses the venv's Python
Ports 8080 / 8000 already in use	Duplicate windows from earlier runs	Scripts check and report the port; close old windows
Planner was very slow	16k context spilled from GPU to CPU	8k context: 100% on GPU
AI proposed 300 s experiments	No limits in the output format	Min/max in the JSON schema and prompt; validation
Website 'fine' while agent said fault	Faults ended quickly; nothing visible	Hold mode: faults stay until rollback
Dashboard flicker, confusing counts	Streamlit full reruns; cumulative counts	AJAX dashboard; 15 s 'active' window; clear labels
/status took 3.7 s	Many kubectl calls per request	Batched find_faults + background watcher (about 9 ms)
Requests 'lost' during slowness	Hanging requests were not counted	In-flight tracking: 'N requests hanging'
Thresholds tripped on noise	Baseline p95 26 ms, 2x is laptop jitter	Floors: 250 ms abort, 100 ms recovery
Rollback said 'not recovered'	Checked before gRPC clients reconnected	Wait for healthy telemetry; backoff cap in frontend
Default target broke whole site	Default was frontend	Default cartservice; agents exclude frontend
Everything crashed for one fault	Original frontend had no fallbacks/timeouts	Resilient Go frontend
Crash fault flickered	Pods restart in about 1 s	1 s crash loop while held; freeze to 0 replicas
redis-cart crash showed 0 impact	Errors name cartservice	Credit redis-cart; forbid splitting the pair
Mistral 7B would not fit	4.7 GB free disk	Ministral 3B (2.75 GB)
Analyst 401 Unauthorized	Cloud model needs ollama signin	Local gemma3:4b as analyst
Squad refused (unhealthy)	Model loading slowed the laptop briefly	Wait up to 45 s for healthy windows
Same picks every run	Fixed roles, low variety	Shuffled roles, random shortlists, seeds
Old findings shown when idle	State kept after rollback	Findings only for the active run
Map not visible	At the bottom of a long page; tiny	Live map on top of every page, flatter layout
'Critical' label with score 36	Model inflated labels	Label from the measured score
Only 48 requests measured	Slow faults slow the probe	At least 150 requests before analysis
No automatic safety net in hold mode	Hold mode only logged breaches	Emergency guardrail (90% for 30 s)

14. Results, limitations, next steps

Key numbers

Measure	Value
Baseline	1,940 requests in 60 s, 0% errors, p50 12.8 ms, p95 26.1 ms
Isolation	9 of 9 services tested broke only their own area on the resilient frontend
Recovery after rollback	9-33 s, verified from telemetry every time
Agent decision time	about 6 s (llama3.2, ministral) to 45-70 s (qwen3 with thinking)

Measure	Value
Analyst	about 13 s per fix with gemma3:4b
Worst run impact	54.5% of requests failed (productcatalogservice outage)
Resilience scores seen	46 (catalogue outage run) and 71 (shipping / currency / payment run)

Limitations (say these before a judge does)

- Small 3-4 B models: actions are safe and valid, but reasons can be generic.
- Laptop scale: one model at a time, so choosing takes 1-2 minutes; single replicas, so no failover is tested.
- Attribution is a best guess when several faults hit the same page; masking is reported, not corrected.
- The severity weights are our design choice; they are open and shown in the UI.
- About 30-90 s of traffic per analysis, so scores vary a little between identical runs.

Next steps

- More fault types: CPU and memory pressure, packet loss, DNS failure.
- Multiple replicas to test failover and autoscaling.
- Learn from past runs to steer agents towards untested weak spots.
- Close the loop: apply a recommended fix automatically in the sandbox and re-run to prove the score improves.

15. Demo script (about 3 minutes)

```
.\start-dashboard.ps1        (opens shop, backend, control center; starts telemetry)
```

Time	Do	Say
0:00	Show the shop, then the control center with the live map all green	A real 11-service shop on Kubernetes; healthy baseline 0% errors, p95 26 ms
0:20	Click Launch 3 agents	Three different models; roles and shortlists are random each run; every pick is validated
0:40	Point at Agent A's streaming reasoning and the map turning amber	The model can only name a service from its shortlist - the schema enforces it
1:20	Errors injected: map turns red, KPIs move, refresh the shop	Only the affected features break - our resilient frontend degrades gracefully
1:50	Ranking and fixes appear; open the Findings page	Measured impact, transparent formula, analyst explanation, resilience score
2:20	Press Kill switch, then Roll back	Freeze vs restore; recovery is verified from live telemetry, not assumed
2:50	Show History and the guardrail card	Full audit trail; automatic rollback if the shop is ever essentially dead

Judge Q&A preparation

Q: Did you train the model?

No. We use pre-trained open-weight models. Our contribution is the system around them: context, constrained output, validation, measurement and safety.

Q: What stops the AI from doing something dangerous?

It never runs commands. It returns a small JSON object whose target must be one of a shortlist; deterministic code validates it, and only fixed fault templates touch the cluster, only in the sandbox namespace.

Q: Why three different models?

Different makers, different reasoning styles, more varied experiments; and it shows the design is model-agnostic.

Q: How is severity decided?

A transparent formula from measured impact (40/25/20/15 weights), then the analyst ranks and explains, but cannot contradict clearly measured differences; labels come from the score.

Q: What does the resilience score mean?

How well the shop kept working for shoppers during this run's errors; 100 - (0.6 x worst + 0.4 x average).

Q: Why not LangChain?

Fewer moving parts, fully auditable, and Ollama's structured output already gives reliable answers.

Q: What happens if something goes badly wrong?

Kill switch freezes; rollback restores and verifies; the guardrail rolls back automatically after 30 s of a near-total outage; a 10-minute backstop removes any forgotten fault.

Q: Is it realistic?

It runs Google's reference microservices app on real Kubernetes with real network faults; the findings (no cart persistence, missing fallbacks, masking) are genuine.

Q: What would you do with more time?

More fault types, replicas and failover, learning from history, and automatically testing the recommended fixes.

16. Glossary

Term	Meaning
Microservice	A small program doing one job (cart, payments...) that talks to others over the network
Kubernetes	Software that runs and restarts containers across machines; here inside one Docker container via kind
Pod / deployment / replica	A pod is a running copy of a service; a deployment says how many copies (replicas) to keep
gRPC	A fast way for services to call each other; Online Boutique uses it internally
Baseline	Measurements of the healthy system, used as the reference
p95 latency	95% of requests finished faster than this
Telemetry	Measurements collected while the system runs
Graceful degradation	Still working with reduced features instead of failing completely
Blast radius	How much of the system one failure affects
Structured output	Forcing a model to answer in an exact JSON format
Kill switch	Instantly stop and freeze
Rollback	Return to the initial state and verify recovery
Guardrail	An automatic limit that triggers a safety action
Ollama	Software that runs AI models locally